32. Birim Testlerini AI ile Hızlandırmak: Jest, PyTest ve Mocklama

Gerçek Veritabanına veya API'ye Bağlanmadan Test Yazma Sanatı
Bölüm 6 • Zorluk Derecesi: Orta Seviye • Tahmini Okuma Süresi: 12 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Unit test koşarken gerçek veritabanına bağlanmak veya gerçek kredi kartından para çekmeye çalışmak neden bir felakettir ve 'Mock' kavramı sistemi nasıl kurtarır?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Birim testler tamamen deterministik ve dış dünyadan (ağ, disk, saat) yalıtılmış (izole) olmalıdır.
> - 📌 Mocklama (Mocking), dış bağımlılıkları (Stripe API, PostgreSQL, E-posta servisi) sahte ama kontrollü nesnelerle değiştirmektir. - 📌 AI modelleri, karmaşık servislerin mock objelerini ve test tablolarını (fixture) saniyeler içinde üretebilir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Tıp fakültesinde ameliyat eğitimi alan bir cerrah adayını düşünün.

İlk gün profesör öğrencinin eline neşteri verip: *"Hadi sedyedeki gerçek hastanın kalbini aç ve ameliyat et!"* der mi? Tabii ki hayır! Öğrenci önce silikon mankenler ve simülasyon cihazları üzerinde yüzlerce kez dikiş atar.

Yazılımda **Mock (Sahte Obje)** işte o silikon mankendir!

Eğer unit test yazarken gerçek Stripe API'sine gidip gerçek kredi kartından 100 dolar çekmeye kalkarsanız, test her çalıştığında şirket batar! Mock kullanarak Stripe'ın döndüğü cevabı taklit eder ve kodunuzun mantığını güvenle doğrularsınız.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Mocklama Mimarisi:

```
[Test Ortamı (Jest / PyTest)]
              |
              ▼
[Sipariş Servisi (Test Edilen Kod – SUT)]
              |
      ┌───────┴────────┐
      ▼                ▼
[GERÇEK DÜNYA]   [MOCK DÜNYASI]
❌ Canlı DB      ✅ jest.fn().mockResolvedValue({ id: 1, balance: 1000 })
❌ Canlı Stripe  ✅ jest.spyOn(stripe, 'charge').mockResolvedValue({ success: true })
(Çok yavaş,      (Işık hızında, sıfır maliyet, %100 kontrol)
bağlantı kopar)
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

````
# ROL: Test Mühendisi ve Jest Gurusu
Aşağıdaki ödeme servisi fonksiyonu için Jest birim testleri yazacağız.

## SERVİS KODU:
```typescript
export async function payInvoice(invoiceId: string, stripeClient: StripeSDK, db:
Database) {
  const invoice = await db.findInvoice(invoiceId);
  if (invoice.isPaid) throw new Error("Fatura zaten ödenmiş.");

  const charge = await stripeClient.charges.create({ amount: invoice.amount });
  await db.markAsPaid(invoiceId, charge.id);
  return { success: true, chargeId: charge.id };
}
````

GÖREV:

1. `stripeClient` ve `db` nesnelerini `jest.fn()` kullanarak mockla (sahte nesneler üret).
2. Başarılı ödeme durumunu test et (`markAsPaid` fonksiyonunun doğru argümanlarla çağrıldığını doğrula).
3. Fatura zaten ödendiğinde Stripe'a ASLA çağrı yapılmadığını (`expect(stripeClient.charges.create).not.toHaveBeenCalled()`) kanıtla.
````

````
---

## 💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

#### ❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):
```typescript
// Kötü Test: Dış bağımlılıkları mocklamadan gerçek veritabanı beklemek
it("faturayı öder", async () => {
  // Eğer PostgreSQL sunucusu açık değilse test patlar!
  const res = await payInvoice("123", new RealStripe(), new RealDB());
  expect(res.success).toBe(true);
});
````
````

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```
import { payInvoice } from "./paymentService";

describe("payInvoice - Birim Test Paketi (Mocklanmış)", () => {
  let mockDb: any;
  let mockStripe: any;

  beforeEach(() => {
    // Her test öncesi temiz mock nesneleri oluşturulur
    mockDb = {
      findInvoice: jest.fn(),
      markAsPaid: jest.fn().mockResolvedValue(true)
    };
    mockStripe = {
      charges: {
        create: jest.fn().mockResolvedValue({ id: "ch_mock_123" })
      }
    };
  });

  it("ödenmemiş faturayı başarıyla tahsil edip veritabanını günceller", async () =>
{
    mockDb.findInvoice.mockResolvedValue({ id: "inv_1", amount: 1500, isPaid:
false });

    const result = await payInvoice("inv_1", mockStripe, mockDb);

    expect(result.success).toBe(true);
    expect(result.chargeId).toBe("ch_mock_123");
    // Yan etkilerin doğrulanması (Assertion of side-effects)
    expect(mockStripe.charges.create).toHaveBeenCalledWith({ amount: 1500 });
    expect(mockDb.markAsPaid).toHaveBeenCalledWith("inv_1", "ch_mock_123");
  });

  it("zaten ödenmiş faturada Stripe çağrısı yapmadan hata fırlatmalıdır", async ()
=> {
    mockDb.findInvoice.mockResolvedValue({ id: "inv_2", amount: 500, isPaid:
true });

    await expect(payInvoice("inv_2", mockStripe, mockDb)).rejects.toThrow("Fatura
zaten ödenmiş.");

    // GÜVENLİK KONTROLÜ: Stripe asla çağrılmamalıdır!
    expect(mockStripe.charges.create).not.toHaveBeenCalled();
    expect(mockDb.markAsPaid).not.toHaveBeenCalled();
  });
});
```

💡 **Kod Analizi:** Bu testler ne internet bağlantısına ne de canlı bir veritabanına ihtiyaç duyar. 1000 adet böyle test toplam 1.5 saniyede koşar ve tüm iş kurallarını %100 doğrular.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Testler arasında mock durumlarını sıfırlamayı unutup testlerin birbirini kirletmesi (Flaky Tests).
  - **Çözüm:** Her testten önce `beforeEach(() => jest.clearAllMocks())` çalıştırın.
- **Tuzak:** Test edilen kodun mantığını değil, mock nesnesinin kendisini test etmek.
  - **Çözüm:** Mock sadece girdiyi sağlar; test ettiğiniz şey kendi yazdığınız servis fonksiyonudur.
- **Tuzak:** Mock kütüphanesinin gerçek API ile uyumsuz hale gelmesi (Stale Mock).
  - **Çözüm:** TypeScript interface'leri kullanarak mock nesnelerin gerçek tipleri implemente etmesini zorunlu kılın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 32: Birim Testleri & Mocklama - Jest ve PyTest Ustalığı*

- 🖥️ **Slayt 1:** Determinizm kuralı: Unit test neden dış dünyadan izole olmalıdır?
- 🖥️ **Slayt 2:** Mock, Stub ve Spy kavramları arasındaki farklar.
- 🖥️ **Slayt 3:** Jest ile bağımlılık enjeksiyonunu (DI) mocklamak.
- 🖥️ **Slayt 4:** Canlı Demo: 1000 testi 2 saniyede koşan mock altyapısı kurmak.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Bir hava durumu API'si çağıran fonksiyon yazın. İnterneti kapatın (offline moda geçin). Jest ile API yanıtını mocklayarak testin internet olmadan 5 milisaniyede geçtiğini canlı konsolda gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizde harici bir API (hava durumu, ödeme veya e-posta) çağıran bir servisi bulun ve Jest mock (`jest.fn()`) kullanarak izole bir test yazın.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Birim testlerde 'Mock' kullanmanın birincil teknik gerekçesi nedir? - A) Kodun satır sayısını artırmak - B) Ağ, veritabanı veya üçüncü taraf API gibi yavaş, maliyetli ve güvenilmez dış bağımlılıkları sahte ama kontrollü nesnelerle ikame ederek testleri hızlı ve deterministik kılmak - C) Sadece Python dilinde geçerli bir zorunluluk olması - D) CSS hatalarını engellemek > **Doğru Yanıt:** B > **Açıklama:** Mock nesneler dış bağımlılıkları yalıtır; testlerin internet veya veritabanı olmadan mikrosaniyeler içinde tutarlı çalışmasını sağlar.

Soru 2: Jest testlerinde `expect(mockFunction).not.toHaveBeenCalled()` assertion'ı neyi doğrular? - A) Fonksiyonun çöktüğünü - B) Belirtilen sahte fonksiyonun test senaryosu boyunca hiçbir koşulda tetiklenmediğini (istenmeyen yan etkinin oluşmadığını) - C) Bilgisayarın kapandığını - D) Testin başarısız sayılması gerektiğini > **Doğru Yanıt:** B > **Açıklama:** Bu ifade, istenmeyen durumlarda (örn. geçersiz şifre veya ödenmiş fatura) kritik fonksiyonların (örn. para çekme) asla çağrılmadığını garanti altına alır.